

Lab 07 - Capstone Game Iteration Studio

What are you making?

Your fixed trio will turn today's rough game prototype into three increasingly trustworthy versions:

Version	Meaning	Minimum visible result
V0 - Prototype	What you have today	A sketch or rough program that communicates the game idea. It may be incomplete.
V1 - Playable core	Monday first build	Start, make one meaningful choice, reach success or failure, and restart.
V2 - Tested iteration	Monday second build	V1 plus explicit contracts, at least six focused tests, and one real red-to-green repair.
V3 - Release candidate	Wednesday build	V2 plus one playtest-driven rule change, regression evidence, a clean run, and a short demo.

The goal is not a large or beautiful game. The goal is a small game that your trio can run, test, debug, review, explain, and improve.

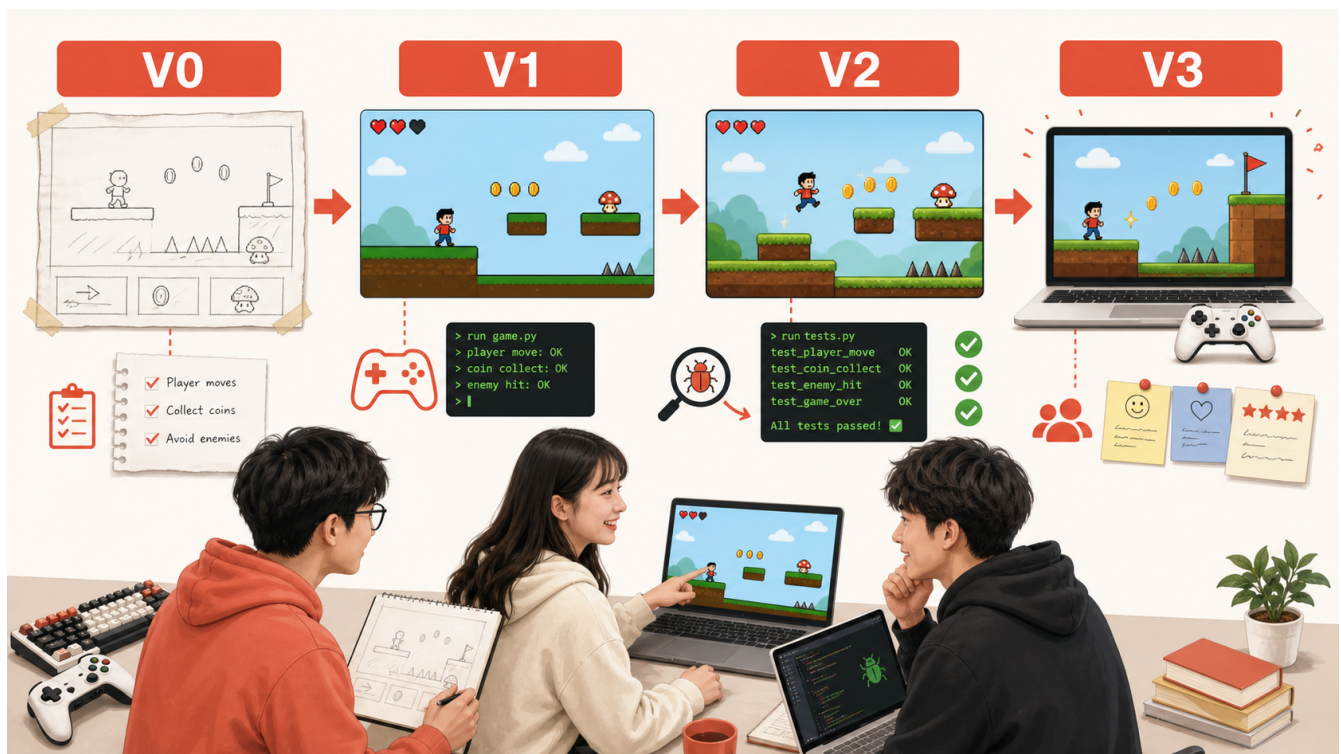


Figure 1. The trio preserves V0, completes a playable V1, proves V2 with tests and debugging, then uses human feedback to prepare V3.

Why this Lab exists

A first prototype only proves that an idea can be shown. It does not yet prove that the rules are clear, the state is reliable, the team can reproduce the run, or another person can review the work. This studio teaches one repeatable engineering cycle:

promise -> contract -> smallest implementation -> focused test -> playtest -> debug -> review -> next version

Each version must leave evidence. New features do not compensate for a broken core, missing tests, fabricated output, or work that only one member understands.

Beginner-safe scope ceiling

Your minimum game needs only:

- 1 one controllable actor, cursor, object, or player choice;
- 2 one state that changes after an action;
- 3 one success or failure condition;
- 4 one restart path; and
- 5 one reproducible run command.

Art, audio, menus, multiple levels, networking, accounts, procedural generation, databases, and advanced AI are optional only after V3 is green. If your current idea is larger, cut it down before adding code.

Trio roles

- Driver: types the agreed small change and narrates what is changing.
- Navigator: reads the contract, protects scope, and predicts the next state.
- Tester / Recorder: states expected observations, runs tests, records evidence, and maintains the AI-use log.

Rotate roles after every accepted Pull Request or at least once per studio block. Every member must author work, review another member's work, and complete an individual explain-back.

L1 AI collaboration contract

L1 is allowed during the studio. AI may:

- explain a Python, Pygame, testing, or Git concept;
- ask guiding questions;
- suggest counterexamples, test ideas, or debugging observations;
- explain a traceback or unfamiliar API documentation; and
- help the trio compare two small design options.

AI may not silently deliver the final project, invent playtest evidence, impersonate a member, approve its own Pull Request, or supply a change the trio cannot explain. Record every material use in `evidence/AI_USE.md`.

The game-neutral interface

Keep your existing prototype. Add a small testable boundary around its core rules, or adapt these names to equivalent functions and document the mapping:

```
def new_game(seed: int = 0):
    """Return a fresh game state with no display or device dependency."""

def apply_action(state, action: str):
    """Return the next state without silently changing the supplied state."""

def is_terminal(state) -> bool:
    """Return True only when the documented success/failure rule is met."""

def snapshot(state):
    """Return stable, comparable evidence for tests and debugging."""
```

The exact state type may be a dataclass, dictionary, tuple, or small object. The contract matters more than the name. Do not rewrite a working prototype merely to copy this interface; write the smallest adapter that makes one rule testable.

Step-by-step studio route

Step 00 - Understand the finish line

DO: Point to V0, V1, V2 and V3 in the illustration. Every member explains the minimum playable loop in their own words.

EXPECT: The trio agrees that V3 is a small, tested release candidate, not a large feature-complete game.

SAVE: No file yet; agree on the definition of done.

STOP AND ASK: A member cannot explain START, ACTION, CHANGE, END and RETRY.

Step 01 - Preserve V0 before changing it

DO: Run or show today's prototype. Record the run command, working behavior and most important limitation in GAME_BRIEF.md. Save one screenshot/photo. Commit and tag it.

```
git add <prototype-and-evidence-files>
git diff --staged
git commit -m "Preserve V0 prototype baseline"
git tag v0-prototype
```

EXPECT: The trio can return to the exact observed baseline.

SAVE: GAME_BRIEF.md, evidence/v0-prototype.png or .jpg, tag v0-prototype.

STOP AND ASK: The prototype cannot be located, run or honestly described.

Step 02 - Write the trio working contract

DO: Complete GROUP_CONTRACT.md. Name all three members, roles, response rule, PR rule, disagreement route, help threshold and L1 AI boundary.

EXPECT: Every member has repository access and a first responsibility.

SAVE: Signed GROUP_CONTRACT.md.

STOP AND ASK: A member has no access, role or agreement about review.

Step 03 - Cut the game to one loop

DO: Fill the START, ACTION, CHANGE, END and RETRY rows in GAME_BRIEF.md. Move at least three tempting features to the exclusions list.

EXPECT: The trio can demonstrate the whole promise without optional art, audio, menus or extra levels.

SAVE: Completed minimum loop and scope ceiling.

STOP AND ASK: The game still requires multiple systems before any complete round is playable.

Step 04 - Expose one testable core boundary

DO: Identify or add equivalents of new_game, apply_action, is_terminal and snapshot. Use starter/src/capstone_contract.py as comments and interface guidance, not as a required rewrite.

EXPECT: One rule can be called without opening a display or asking a human to press a key.

SAVE: Actual interface mapping in evidence/CONTRACTS.md.

STOP AND ASK: Testing one rule still requires running the entire frame loop.

Step 05 - Build V1 start and action

DO: Create a short branch. Make the game start and accept one meaningful action. Review the staged diff before committing. Open a Pull Request reviewed by another member.

```
git pull --ff-only
git switch -c v1-player-action
# edit one contract-sized behavior
python -m pytest -q <focused-test-path>
git add <only-intended-files>
git diff --staged
git commit -m "Implement V1 player action"
git push -u origin v1-player-action
```

EXPECT: One action creates one visible, explainable state change.

SAVE: PR URL, focused observation and role row in evidence/ITERATION_LOG.md.

STOP AND ASK: The branch changes unrelated systems or no member can explain the state transition.

Step 06 - Complete V1 end and restart

DO: Rotate roles. Add one reachable success/failure rule and restart. Ask a second member to run the loop without author coaching.

EXPECT: START -> ACTION -> CHANGE -> END -> RETRY is playable.

SAVE: V1 screenshot, peer-run observation and tag v1-playable.

STOP AND ASK: Restart requires manually editing files or relaunching the development environment.

Step 07 - Write four contracts and six test expectations

DO: Complete evidence/CONTRACTS.md and evidence/TEST_PLAN.md before writing many tests. Replace every skipped template with a project-specific test.

EXPECT: Tests cover fresh/restart, normal action, boundary/invalid action, terminal rule, invariant and regression.

SAVE: Four contracts, seven-row test plan and test files.

STOP AND ASK: A test expectation cannot be stated until after the code runs.

Step 08 - Capture one genuine red-to-green repair

DO: Choose a real failing test or observable defect. Fill evidence/DEBUG_LOG.md while it is red. Change the smallest contract-owned boundary, then rerun focused and full tests.

```
python -m pytest -q <one-focused-test>
python -m pytest -q
git diff
```

EXPECT: The focused test changes red-to-green and prior tests remain green.

SAVE: Debug log, fixing PR and exact command observations.

STOP AND ASK: The proposed repair edits multiple unrelated rules or the team is reconstructing a fake failure after success.

Step 09 - Freeze V2 tested iteration

DO: Confirm at least six meaningful tests, zero required skips and one reviewed repair. Run the whole suite and tag the accepted commit.

```
python -m pytest -q
git status --short
git tag v2-tested
```

EXPECT: The suite is green and the working tree is clean.

SAVE: Exact test summary and tag v2-tested.

STOP AND ASK: Tests pass only on one uncommitted machine or required templates remain skipped.

Step 10 - Conduct one silent human proxy playtest

DO: Give only the documented starting instruction. Do not coach. Record exact actions/quotes in evidence/PLAYTEST.md, then separate observations from interpretations.

EXPECT: The trio identifies one player-blocking or rule-level issue from evidence.

SAVE: Anonymous playtest record and one selected issue.

STOP AND ASK: The change is based only on team preference or a synthetic AI persona rather than a human observation.

Step 11 - Build and verify V3

DO: Turn the selected issue into an acceptance check. Implement one small change, add/update a focused test when feasible, rerun regression and repeat the relevant playtest action.

EXPECT: Evidence shows improved, unchanged or worse behavior honestly; prior core rules still pass.

SAVE: V3 PR, repeat observation, known limitation and tag v3-release-candidate.

STOP AND ASK: The team starts a feature list, changes the test to excuse a defect, or cannot state what observation would disprove improvement.

Step 12 - Rebuild, demonstrate and seed the report

DO: Clone to a clean folder or second computer, install, run tests and launch the game. Rehearse a two-minute demo. Complete FINAL_REPORT_OUTLINE.md and individual explain-backs.

```
python -m pytest -q
python -m your_game
git status --short
git log --online --decorate -12
git tag --list 'v*'
```

EXPECT: Another person can reproduce V3; every member explains one state decision, test and repair/review decision.

SAVE: Clean-run receipt, report evidence map and final commit hash.

STOP AND ASK: Only one member can run or explain the submitted game.

Monday studio outcome

By the end of Monday, the trio should have:

- preserved V0;

-
- a complete working contract and bounded MVP;
 - V1 playable core;
 - V2 with at least six focused tests;
 - one genuine debug record;
 - reviewed Pull Requests from all three members; and
 - tags v0-prototype, v1-playable, and v2-tested.

Wednesday studio outcome

By the end of Wednesday, the trio should have:

- one human playtest record;
- one evidence-driven rule change;
- V3 regression evidence;
- a clean-clone installation/run receipt;
- tag v3-release-candidate;
- a two-minute demo route;
- a completed final-report outline; and
- one individual explain-back from every member.

GitHub workflow

Use one private repository. Never share passwords or tokens.

```
git pull --ff-only
git switch -c v1-player-action
# make one small change and run its focused test
git add <only-intended-files>
git diff --staged
git commit -m "Implement one playable player action"
git push -u origin v1-player-action
```

Open a Pull Request. A different member reviews the contract, diff, test observation, and AI-use record before merge. Pull main before starting the next branch.

Acceptance commands

Adapt commands to the repository, record the exact commands used, and do not invent output:

```
python -m pytest -q
python -m your_game
git status --short
git log --oneline --decorate -12
git tag --list 'v*'
```

Stop and ask for help when

- the project cannot be cloned or started on a second computer;
- the trio cannot reduce the idea to one playable loop;

-
- test collection fails before a test runs;
 - the failing behavior is outside the current contract;
 - two branches modify the same large file without an integration plan;
 - 15 focused minutes produce no new observed fact; or
 - AI proposes code no member can explain.

Learning links

- Python functions: <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
- Python data structures: <https://docs.python.org/3/tutorial/datastructures.html>
- pytest first test: <https://docs.pytest.org/en/stable/getting-started.html>
- Pygame tutorials: <https://pyga.me/docs/tutorials/en/index.html>
- GitHub flow: <https://docs.github.com/en/get-started/using-github/github-flow>
- Reviewing Pull Requests:
<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests>
- Writing useful issues:
<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue>